



Threat Detection with GuardDuty



Kehinde Abiuwa

The screenshot shows the AWS GuardDuty console interface. On the left, there's a navigation menu with options like Summary, Findings, EC2 malware scans, Protection plans, and Accounts. The main area displays a single finding with a title: "Credentials for the EC2 instance role NextWork-GuardDuty-project-kehindebiuwa-TheRole-JX8ffxnWDFC were used from a remote AWS account." The finding is marked as "High" severity and "First seen 21 minutes ago, last seen 21 minutes ago". Below the title, there's a description: "Credentials created exclusively for an EC2 instance using instance role NextWork-GuardDuty-project-kehindebiuwa-TheRole-JX8ffxnWDFC have been used from a remote AWS account 133743800556." There's a link to "Investigate with Detective". Below this, there are buttons for "Useful" and "Not useful". An "Overview" section follows, showing details like Finding ID, Type (UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration:InsideAWS), Severity (HIGH), Region (eu-west-1), Count (1), Account ID (511239431868), and Resource ID (nextwork-guardduty-project-kehinde-thesecurebucket-xnftyslnu8rx).

Overview	
Finding ID	f4cc886bcd050cc2998bb70e14f0beca
Type	UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration:InsideAWS
Severity	HIGH
Region	eu-west-1
Count	1
Account ID	511239431868
Resource ID	nextwork-guardduty-project-kehinde-thesecurebucket-xnftyslnu8rx
Created at	09.05.2025 10:20:40 (16 minutes ago)



Introducing Today's Project!

Tools and concepts

The services I used were Amazon GuardDuty, Amazon CloudFront, Amazon S3, AWS CloudFormation, IAM, and AWS CloudShell. Key concepts I learnt include simulating SQL injection and command injection, stealing and using instance role creds via IMDSv2 with a separate CLI profile, analyzing GuardDuty findings (stolen-credential use and Object:S3/MaliciousFile), checking activity in GuardDuty, enabling Malware Protection and testing with EICAR, and hardening with parameterized queries, least-privilege IAM, key rotation/revocation, and IMDS/WAF controls.

Project reflection

This project took me approximately 63 minutes. The most challenging part was fixing the CloudFormation stack (ASG launch config/region mismatch) and then wiring the stolen CLI profile with the temporary creds. It was most rewarding to see GuardDuty fire the stolen-credential finding and the S3 malware finding after the EICAR upload—clear proof my detection pipeline worked end to end.

placeholder

I did this project to showcase my skills in AWS cloud security and threat detection—spinning up a lab with CloudFormation, simulating SQL/command injection and credential theft, using CloudShell to exfiltrate from S3, and then detecting, investigating, and responding with GuardDuty, CloudTrail, IAM hardening, and Malware Protection—all clearly documented for my portfolio.



Project Setup

To set up for this project, I deployed a CloudFormation template that launches an intentionally vulnerable web app (OWASP Juice Shop) fronted by an ALB + CloudFront, a “secure” S3 bucket seeded with sensitive data, and GuardDuty with an auto-remediation workflow. The three main components are: Web app stack — an EC2 Auto Scaling Group with a Launch Template that installs Juice Shop + Nginx, sitting behind an Application Load Balancer and exposed through CloudFront. Data target — an encrypted S3 bucket with a policy denying insecure transport and a VPC endpoint; the instance startup script uploads a “secret-information.txt” file to this bucket. Detection & response — a GuardDuty detector (multiple features enabled) plus an EventBridge rule and Step Functions state machine that react to credential-exfiltration findings by attaching a deny policy to the instance role.

The web app deployed is called OWASP Juice Shop. To practice my GuardDuty skills, I will safely simulate suspicious behavior against my own app, watch GuardDuty generate findings, investigate them in the console, and then respond so I can see end-to-end detection and remediation in action.

GuardDuty is AWS’s managed threat detection service that continuously analyzes AWS signals to spot suspicious activity, known bad indicators, and anomalies, then raises actionable findings. In this project, it will detect and alert on things like credential misuse (anomalous API calls, unusual S3 reads), network reconnaissance/command-and-control, and potential malware on the instance (via on-demand EBS snapshot scanning). We’ll review those findings in the console and use them to practice investigation and response.



The screenshot displays the AWS CloudFormation console interface. On the left, the 'Stacks' tab is active, showing a single stack named 'NextWork-GuardDuty-project-kehindeabiwa' with a status of 'CREATE_COMPLETE'. On the right, the 'Events' tab is active, displaying a list of 88 events. The events table includes columns for 'Timestamp', 'Logical ID', and 'Status'. The first event is a 'CREATE_COMPLETE' for the stack itself, followed by events for resources like 'CloudFrontDistribution' and 'TheAutoScalingGroup'.

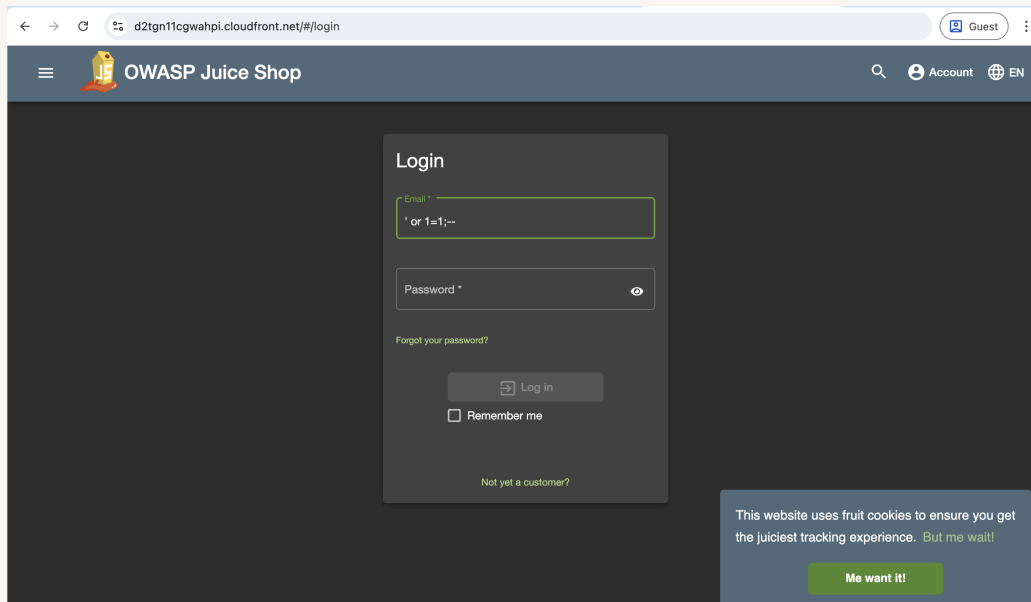
Timestamp	Logical ID	Status
2025-09-03 17:48:48 UTC+0100	NextWork-GuardDuty-project-kehindeabiwa	CREATE_COMPLETE
2025-09-03 17:48:47 UTC+0100	CloudFrontDistribution	CREATE_COMPLETE
2025-09-03 17:48:00 UTC+0100	TheAutoScalingGroup	CREATE_COMPLETE
2025-09-03 17:48:00 UTC+0100	TheAutoScalingGroup	CREATE_IN_PROGRESS



SQL Injection

The first attack I performed on the web app is SQL injection, which means sending crafted input that gets merged into an app's SQL query so the database executes unintended commands (e.g., turning a login check into "always true"). SQL injection is a security risk because it can bypass authentication, leak or alter data, delete records, run admin operations, and ultimately compromise the whole app. Mitigations include parameterized queries/prepared statements, strict input allow-listing, least-privilege DB accounts, and protective layers like WAFs and good error handling.

My SQL injection attack involved putting text in the login field that made the app's database query always true, letting me in without a real password. This means the app was building SQL by concatenating user input instead of using parameterized queries, so the check was bypassed.

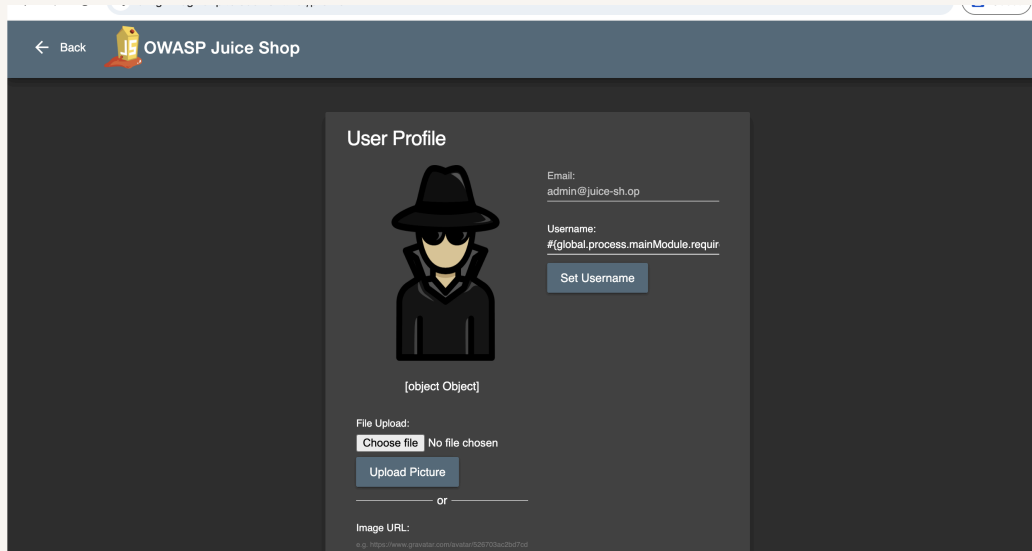




Command Injection

Next, I used command injection, which is when an app takes user input and runs it as a system command on the server. The Juice Shop web app is vulnerable to this because the profile "username" field is processed by a server-side template that evaluates what you type instead of treating it as plain text, letting my input reach Node.js's command runner and pull instance credentials.

To run command injection, I pasted a template expression into the Profile → Username field so the server evaluated it and executed a system command (via Node's `child_process.exec`). The script will request an IMDSv2 token, pull the instance's temporary IAM credentials from the metadata service, and save them to a public `credentials.json` file so I can view the stolen creds in the browser.





Attack Verification

To verify the attack's success, I opened the CloudFront URL my payload wrote to —/assets/public/credentials.json—and checked the file. The credentials page showed me a JSON with an AccessKeyId, SecretAccessKey, Session Token, and Expiration (Code: "Success"), confirming the server ran my command and exfiltrated the instance's temporary IAM credentials.

```

{
  "AccessKeyId": "ASIAXC6U26EX40XUTM",
  "Code": "Success",
  "Expiration": "2025-09-03T23:46:22Z",
  "LastUpdated": "2025-09-03T17:20:17Z",
  "SecretAccessKey": "TSeIM72jHSs/7F7ECj f6FF5AgDopnahasah7Q7Xl",
  "Token": "
IQA0b3pZ2LwX2VjEOL/////////wEaCWV1LXd1c30tMSJHMEUCIDou/Pa0hvqW+Fbgte+ahINwP4+9dwB5wif4t4CvSFIiAiEAo1yw9IZuTFaUmXsZT4A+Tz3Aqa6V55ini6Wky7HuPpUquAUIShAAGgw1MTEyMz
k0MzE4NjgiDNHU9X3/NM6AupjjsQVBQij4uqLTW42mPo6C//VAqXvCTXtABe54KHzc/ySwa6yTaaYKSt/gZM7d/D6TRkq05f3MYStyk082JGy1QF/BLBDTn7/Ww+pHhurLoA+itzro83wHByBuUMLi4k/SS5t9bAw
1Zjt89NA5jU0iJN2Q3KDvN07/s9MQVY4jXLogL+L4vj t880L1GnED2mpP9xTw7chs/hQhE4glztk178otdgdwgi1bU0HUosr7TivgU85A+yf03YuxvEMsfrZVz7zKe8FEbNyT5dyGPKt5vRL1GF0mxPdaUJRhcV3e4u
0Wtpq3mncU5v4qht40tp8xt7s83Iwc1GeeGRvI/Kk)x7xmkAs3Q5g5StgswTULn017Gw20qBUf/f3+g8B1VcBLfZkAIDR0xq1h08J8eUfKbVgpFh+Dc0Wz39ahR65xYQH1/iCnTU0SXZem2CLE5pdxEZ72Ny4hS
+15xcx13pqa28dyC7CZ2ZlJrVAdEQJbgorIpsIyugXLzbuYmCS36m0sTmtJkTR1LDBtkQVrS/n9LunROBFZCz24dY5W4Gv09Nw+P9mnmjUAMgThVePDCZmStXXSHYz44s1TYLX+wa3RN37Y53P16c5dJN2D06Gg/
IB1QZ/osC6ru0IP+U0Y60yYiU/MZwL12ov9E3htsv+PwoZecqUgTIQGLUWfGfZTSxeoBNG2GLehx1imP1CT1gh3pMA0uh50HZINOjsgISU9ck34ndM/U7E4KwalUm0t522U17YwGLCa0gMqyMSyVpBzPK4LYTcyx/
jJ0Uz6C1hUGNe9DL46KX7c8kQ7AUGMqJqI3hdv7VgmU9UfbFCws2B5dNKSg80m/AlvM/SmmULstMv+B5N08Nh2cG2WHVT0YhJgw+/HhxQY6sQGcntId6q0QpEp4T4Kb9tmk00uX0XCfJ81t84VpP/1nmvPDCZ11vPC
SxE6Z5I7n+g15hdsPC1oZzDr1F0h5KkvHONqH7VM8Wfn94/YhCzvtJYzRU+eQ6TvvHAFALCgqGGS0ba4FGW2EkvcofIdIE0D9KjK0XubY80W1P9Zt15NvDK2K6X9vUpBsfxyZuW9yQwPdxDTgn7tYlKCaKwflWPY
57swE19VndUrOPTN62o=",
  "Type": "AWS-HMAC"
}
```



Using CloudShell for Advanced Attacks

The attack continues in CloudShell, because it's a browser-based shell that runs inside my AWS account with the AWS CLI ready to use. I can quickly paste the stolen temporary credentials, run attacker-style commands, and prove access—while keeping everything contained and audited by CloudTrail, with no tools to install or secrets saved on my laptop.

In CloudShell, I used `wget` to download the `stolen_credentials.json` file from the CloudFront URL to my shell. Next, I ran a command using `cat` and `jq` to display and pretty-print the JSON so I could clearly see the `AccessKeyId`, `SecretAccessKey`, `SessionToken`, and `Expiration`—confirming the creds were captured and still valid to configure the AWS CLI.

I then set up a profile, called `stolen`, to load the stolen temporary credentials and run AWS CLI commands as the "hacker" in my own account. I had to create a new profile because I didn't want to overwrite my normal creds—keeping a separate profile isolates the attack session, makes it easy to switch back, and lets GuardDuty/CloudTrail clearly show actions from the stolen creds (and we'll reuse this session later).



```
~ $ aws configure --profile=stolen-aws-session-token --cat-credentials.json --jq --profile=stolen
~ $ aws s3 cp s3://$SOURCE_BUCKET/secret-information.txt . --profile=stolen
download: s3://nextwork-guardduty-project-kehinde-thesebucket-xnftyslnu8rx/secret-information.txt to ./secret-information.txt
~ $ cat secret-information.txt
Dang it - if you can see this text, you're accessing our private information!
~ $ █
```

Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy



GuardDuty's Findings

After performing the attack, GuardDuty reported a finding within a few minutes (about 5 minutes). Findings are near-real-time alerts that describe the suspicious activity, the affected resource, severity, timestamp, and recommended next steps so you can investigate and respond.

GuardDuty's finding was called

"UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.InsideAWS" (shown in the console as "Credentials for the EC2 instance role ... were used from a remote AWS account"), which means credentials meant for an EC2 instance were stolen and then used from another AWS account. Anomaly detection was used because GuardDuty compares activity to normal patterns and flags unusual use—like instance role creds appearing from a different account/identity/location—indicating likely credential theft.

GuardDuty's detailed finding reported that EC2 instance credentials were stolen and used from a remote AWS account. It included: Resource affected: the EC2 instance role that was used to access the Juice Shop S3 bucket. Action: an S3 object read (the secret text file was retrieved). Location/IP: the source location and IP of the attacker, matching my CloudShell session in my AWS Region. This gave me the who/what/where of the attack so I could confirm credential theft, see exactly what was taken, and plan how to prevent it next time.



aws

Search

[Option+S]

Europe (Ireland)

Account ID: 5112-3943-1868

kehindeabiuwa-IAM-Admin

GuardDuty

Summary

Findings

EC2 malware scans

Protection plans

S3 Protection

EKS Protection

Extended Threat Detection New

Runtime Monitoring

Malware Protection for EC2

Malware Protection for S3

RDS Protection

Lambda Protection

Accounts

Usage

Settings

GuardDuty > Findings

Findings (1) Info

Create suppression rule

Actions

Saved rules

Apply saved rules

Filter findings

Status

Current

Threat type

All findings

< 1 >

☐

Title

☐

Credentials for the EC2 instance JX8ffxnWDFFC were used from a

Credentials for the EC2 instance role NextWork-GuardDuty-project-kehindeabiuwa-TheRole-JX8ffxnWDFFC were used from a remote AWS account.

High First seen 21 minutes ago, last seen 21 minutes ago

Credentials created exclusively for an EC2 instance using instance role NextWork-GuardDuty-project-kehindeabiuwa-TheRole-JX8ffxnWDFFC have been used from a remote AWS account 133743800556.

[Investigate with Detective](#)

This finding is Useful Not useful

Overview

Finding ID	f4cc886bcd050cc2998bb70e14f0beca	Q Q
Type	UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration:InsideAWS	Q Q
Severity	HIGH	Q Q
Region	eu-west-1	
Count	1	
Account ID	511239431868	Q Q
Resource ID	nextwork-guardduty-project-kehinde-thesebucket-xnftyslnu8rx	Q
Created at	09.03.2025 19:29:49 (16 minutes ago)	



Extra: Malware Protection

For my project extension, I enabled GuardDuty Malware Protection for S3 so AWS can automatically scan S3 objects for malicious content and raise findings. Malware is malicious software (like viruses, trojans, or ransomware) that can hide on servers or in files and be used to steal data or damage systems.

To test Malware Protection, I uploaded the EICAR antivirus test file (the standard harmless test string) to my S3 bucket so GuardDuty would scan it and raise a malware finding. The uploaded file won't actually cause damage because it isn't real malware—it's a safe pattern specifically designed to trigger antivirus/GuardDuty detections.

Once I uploaded the file, GuardDuty instantly triggered a High-severity "Object:S3/MaliciousFile" finding for the EICAR test file in my S3 bucket. This verified that Malware Protection for S3 is enabled and scanning new objects, correctly flagging the harmless test string (not a real virus) and giving me the object, account/region, and timestamp to respond.



eu-west-1.console.aws.amazon.com/guardduty/home?region=eu-west-1#/findings?macros=current&tid=b8cc8885b3e43ed539554c945dfc8e5c Guest

aws Search [Option+S] Europe (Ireland) Account ID: 5112-3943-1868 kehindeabiwa-IAM-Admin

GuardDuty

Summary
Findings
EC2 malware scans

▼ Protection plans
S3 Protection
EKS Protection
Extended Threat Detection [New](#)
Runtime Monitoring
Malware Protection for EC2
Malware Protection for S3
RDS Protection
Lambda Protection

Accounts
Usage
Settings

GuardDuty > Findings

Findings (2) Info

Create suppression rule

Actions ▼

Saved rules
Apply saved rules ▼

Filter findings

Status: Current Threat type: All findings

1

scan on your S3 object EICAR-test-file.txt has detected a security risk EICAR-Test-File (not a virus).

A malware scan on your S3 object EICAR-test-file.txt has detected a security risk EICAR-Test-File (not a virus).

High First seen 3 minutes ago, last seen 3 minutes ago Info

A malware scan on your S3 object arn:aws:s3::nextwork-guardduty-project-kehinde-thesebucket-xnftyslnu8rx/EICAR-test-file.txt has detected a security risk EICAR-Test-File (not a virus).

[Investigate with Detective](#)

This finding is Useful Not useful

Overview

Finding ID	b8cc8885b3e43ed539554c945dfc8e5
Type	Object:S3/MaliciousFile
Severity	HIGH
Region	eu-west-1
Count	1
Account ID	511239431868
Resource ID	nextwork-guardduty-project-kehinde-thesebucket-xnftyslnu8rx
Created at	09-03-2025 20:26:23 (3 minutes ago)
Updated at	09-03-2025 20:26:23 (3 minutes ago)